

ТЕСТОВОЕ ЗАДАНИЕ

СТАЖИРОВКА ПО НАПРАВЛЕНИЮ

АНАЛИТИК ПРОИЗВОДИТЕЛЬНОСТИ СИСТЕМ

Автор: Хальзев А. И.

[Git-репозиторий](#)

Задание 1:

Продукт 1 – Переводчик и обработчик человеческого языка.

Учитывая современные тенденции, где практически каждый человек сталкивается с общением с NLP моделями искусственного интеллекта, необходим переводчик на условно машинный язык с человеческого и наоборот. В качестве такого языка может выступать язык математической логики, так как, в отличие от человеческого он практически не содержит неточностей и ошибок в своей формулировке.

Язык логики более понятен для компьютера, он очищает текст от лишнего мусора, неважного для компьютера. Это позволит сделать ИИ куда более точным особенно при решении задач, требующих точных ответов, где даже путь к ним будет последовательностью аксиом и правил. Такой дополнительный слой алгоритма не требует долгого обучения на огромных массивах данных, а лишь набор точных аксиом и других правил. Если же в тексте возникнут какие-то неточности, например, омонимы, то компьютер гарантированно будет находить эти неточности и уточнять у пользователя.

Реализовать точный алгоритм такого перевода будет непросто из-за сложности самого человеческого языка, но, к счастью, есть множество готовых дата баз, которые помогут с некоторыми проблемными аспектами, как лемматизация, например.

Нефункциональные требования к производительности:

1. Время перевода запроса – особенно длинные промты будет сложно перевести на язык математической логики, что будет занимать значительное количество времени, поэтому время обработки и перевода алгоритмом будет

важным критерием. На перевод и составление уточняющих вопросов у пользователя не должно превышать 2х секунд.

2. Время обработки запроса – чтобы найти необходимое решение, компьютер начнет перебирать все возможные комбинации фактов, чтобы найти доказательство. В сложных запросах (особенно с кванторами) время ответа может расти экспоненциально. Потому время обработки и выдачи ответа не должно превышать 5ти секунд.

3. Устойчивость к ошибкам пользователя – если пользователь допускает критические ошибки (например, неправильная расстановка запятых, орфографические ошибки), система не должна падать при возникновении противоречия.

Продукт 2 – Общее облачное хранилище фото и видео.

Сервис позволяет нескольким пользователям (например, членам семьи) объединять свои фотографии, видео в общем защищённом пространстве. Система самостоятельно забирает снимки из подключённых источников (личные облачные хранилища, соцсети, устройства), анализирует, отбирает лучшие и раскладывает по альбомам в соответствии с предпочтениями пользователей, например, по дате, месяцу или людям на ней. Это позволит пользователям меньше заморачиваться над сборкой и хранением, а также сохранит фотографии даже в случае их утери.

Нефункциональные требования к производительности:

1. Скорость импорта и обработки фотографий - при подключении нового источника или массовой загрузке, когда, например, пользователь спустя длительное время появляется в сети, система должна обрабатывать не менее 1000 фотографий за 5 минут (включая скачивание, распознавание, классификацию и размещение по альбомам).

2. Эффективный алгоритм поиска и сортировки данных – система должна эффективно сортировать и искать необходимые фотографии из тысяч по заданным фильтрам.

3. Масштабируемость и одновременная работа - Сервис должен поддерживать не менее 1 000 000 активных семейных групп (аккаунтов) и обрабатывать пиковую нагрузку 10 000 одновременных операций импорта или синхронизации в секунду без деградации времени ответа API.

Продукт 3 – Платформа мониторинга городского трафика.

Для размещения эффективной рекламы, планировании мероприятий, составления маршрутов транспорта, расписаний работы магазинов и т.д. был бы полезен сервис для отслеживания человеческого трафика на улицах, как для машин на дорогах. Сервис будет собирать информацию с ближайших камер, использовать данные о положении устройств пользователей и отображать в реальном времени пешеходный трафик на улицах, прогнозировать загруженность в определенные дни, в зависимости от погоды, например.

Нефункциональные требования к производительности:

1. Получение данных - время от получения сырых данных с источника (камера, датчик) до отображения обновлённого значения загруженности на карте не должно превышать 2 секунд.

2. Анализ данных - система должна обрабатывать поток данных от множесва источников, обрабатывать из и анализировать.

3. Отображение данных - API для получения текущей загруженности конкретной зоны или маршрута должен отвечать со временем не превышающем миллисекунд для множества пользователей.

Задание 2:

Часть 1.

Таблица 2.1 – Исходная таблица.

Номер замера	Сетевое устройство	Тактовая частота ЦП, GHz	UDP Throughput, Мбит/с	Утилизация ЦП, %	Температура ЦП, С

1	1	1500	600	50	38
2	1	1500	650	55	45
3	1	1500	700	60	52
4	1	1500	750	65	60
5	1	1500	900	70	67
6	2	2600	750	35	30
7	2	2600	800	40	31
8	2	2600	850	45	32
9	2	2600	900	50	33
10	2	2600	600	30	28

Для решения задачи прогноза значения пропускной способности по протоколу UDP, Мбит/с (определим, как целевую переменную) для сетевого устройства №3 можно использовать линейную регрессию. Использование линейной регрессии допустимо, так как, исходя из таблицы 2.1, линейность зависимости целевой переменной от значений утилизации и температуры ЦП можно легко определить визуально (при изменении значения утилизации ЦП на 5 единиц и температуры на 7, UDP Throughput увеличивается на 50 Мбит/с, похожую зависимость можно увидеть для второго устройства).

Но для пятого замера наблюдается скачок с 750 до 900 Мбит/с для целевой переменной, при условии линейного роста нецелевых переменных. Такое отклонение примем за шум, чтобы не усложнять модель ненужной полиномиальностью, точно так же поступим для 10ого замера, где тоже наблюдается нарушение линейности, хоть она менее существенна.

При проведении промежуточных тестов выяснилось, что модель неправильно описывает данные, полученная модель завышает коэффициент температуры в положительную сторону, как будто температура положительно влияет на работу устройства, что физически неверно. Из этого делаем вывод, что данных просто недостаточно для корректного обучения модели, потому в

приложенном коде будут две версии: с влиянием температуры, что было бы математически верно и без нее, что верно логически.

Для вычисления целевой переменной для Зего устройства с тактовой частотой ЦП 2000 GHz, когда оно находится под нагрузкой, вызывающей утилизацию ЦП 40% и температуру ЦП 54 °C будем использовать Python и библиотеку sklearn для вычислений.

Полученные значения:

Файл model_no_temp.py (без учета температуры в модели):

Прогнозируемый UDP Throughput: **712.08 Мбит/с**

Уравнение: $Y = 0.09 * \text{Частота} + 2.58 * \text{Утилизация} + 434.08$

Файл model_temp.py (с учетом температуры в модели):

Прогнозируемый UDP Throughput: **980.28 Мбит/с**

Уравнение: $Y = 0.27 * \text{Частота} + -5.22 * \text{Утилизация} + 14.18 * \text{Температура} + -109.67$

Часть 2:

Таблица 2.2 – Исходная таблица.

Номер замера	UDP Throughput, Мбит/с	Утилизация ЦП, %	Температура ЦП, C
1	50	?	?
2	100	?	?
3	150	?	?
4	200	?	?
5	250	?	?
6	300	?	?
7	350	?	?
8	400	?	?

9	450	?	?
10	500	?	?

Для решения задачи воспользуемся методом обратной линейной регрессии, так как для этого будем использовать формулы зависимостей нагрузки от трафика для каждого устройства по отдельности:

$$Util_1 = a_{Util1} * UDP + c_{Util1}$$

$$Temp_1 = a_{Temp1} * UDP + c_{Temp1}$$

Где c – свободный член.

Такие формулы применимы, так как температура не является причиной высокой выходной мощности, а лишь следствием. Аналогично для второго устройства.

Для вычисления необходимых предсказываемых данных будем использовать коэффициент интерполяции, который отражает к какому из первых двух устройств ближе четвертое, исходя из значений их частоты.

Полученные значения:

Файл `reverse_regression.py`:

Таблица 2.3 – Полученная таблица.

Номер замера	UDP Throughput, Мбит/с	Утилизация ЦП, %	Температура ЦП, С
1	50	4.3	26.1
2	100	8.7	27.2
3	150	13.0	28.4
4	200	17.4	29.5
5	250	21.7	30.6
6	300	26.1	31.7
7	350	30.4	32.9
8	400	34.8	34.0

9	450	39.1	35.1
10	500	43.5	36.2

Часть 3:

В результате работы над заданием был произведен анализ данных. В таблице 2.1 была выявлена линейная зависимость между пропускной способностью и утилизацией ЦП, его температурой. Также выявлены аномальные замеры, которые отклоняются от тенденции. При проведении анализа была выявлена нелогичность полученных данных: температура оказывала положительный эффект на работу ЦП, это произошло в первую очередь из-за малого количества данных, при большем количестве данных коэффициент температуры предполагался быть отрицательным.

На основе данных таблицы 2.1 был получен прогноз для нового устройства с другими данными. Результаты получились правдоподобными 712.08 Мбит/с и 980.28 Мбит/с в зависимость от отношения к температуре.

Для таблицы 2.1 с помощью метода обратной линейной регрессии были получены искомые данные, базируясь на значениях таблицы 2.2, результаты представлены в таблице 2.3. Данные в полученной таблице растут линейно и соответствуют ожидаемым значениям.

Упомянутые файлы с кодом представлены в приложении А и по ссылке в [Git-репозитории](#).

Задание 3:

1. Метод измерения производительности – совокупность способов или правил оценки качества устройства по одному или нескольким показателям производительности.

Методика нагрузочного тестирования – алгоритм проведения оценки производительности устройства, метод сбора данных, их анализа в соответствии с заявленными требованиями.

2. Исходное НФТ: Проектируемое сетевое оборудование должно обладать пропускной способностью по протоколу UDP не менее 900 Мбит/с, при этом утилизировать ресурсы ЦП не более чем на 80%. Температура ЦП при максимальной производительности не должна подниматься выше 60 °С.

НФТ к производительности продукта необходимо скорректировать и дополнить, готовое НФТ:

Тестируемое изделие Сервер iRU Rock s2208e с тактовой частотой 2,0 ГГц должно быть испытано при температуре воздуха 24 °С; в ходе НФТ генератор трафика Xena Vulcan 100G-4Q создаёт UDP-поток с длиной кадра 1024 байта и количеством параллельных сессий 64. После прогрева в течение 5 минут в установившемся режиме длительностью 5 минут средняя пропускная способность должна составлять не менее 900 Мбит/с при потерях пакетов не более 0,01%, а пиковая утилизация каждого ядра CPU не должна превышать 80%, при этом максимальная температура ЦП в любой момент теста не должна подниматься выше 60°С. Тест проводится 5 раз с полным остыванием между циклами; за итоговый результат принимается наихудший показатель из всех измерений.

Без уточнения подоюных требований, как мне кажется, проведение эксперимента будет бессмысленным, так как в исходном их слишком мало и результаты будут разительно отличаться.

3. а, b

Параметры объекта тестирования (название – принимаемые значения):

1. Модель тестируемого устройства - Сервер iRU Rock s2208e.
2. Модель процессора - Intel Xeon E-2378G.
3. Тактовая частота CPU - 2,0 ГГц.
4. Объём и тип оперативной памяти - 64 ГБ, DDR5-4800 МГц
5. Версия драйверов и операционной системы - Linux Ubuntu 22.04 LTS
6. Температура окружающей среды - 24°С ± 1°С
7. Схема подключения - приём трафика на тестируемое устройство

Параметры методики:

Параметры генератора нагрузки:

1. Модель генератора трафика - Xena Vulcan 100G-4Q
2. Версия ПО управления генератором - Xena Manager v.2.10
3. Тип портов генератора - 2 порта 10 GbE SFP+

Параметры метода нагрузки:

1. Протокол транспортного уровня - UDP
2. Размер кадра - 1024 байта
3. Количество одновременных потоков - 64 потока
4. Длительность этапа измерения - 5 минут
5. Количество циклов тестирования – 5

3. с

Метрики производительности:

1. Средняя пропускная способность - Среднее арифметическое значение скорости передачи полезных данных за весь период измерения (≥ 900 Мбит/с)
2. Пиковая пропускная способность - Максимальное мгновенное значение скорости, зафиксированное за период измерения
3. Коэффициент потерь пакетов - Отношение потерянных пакетов к общему числу отправленных пакетов ($\leq 0,01\%$)
4. Средняя утилизация CPU - Среднее арифметическое значение загрузки всех логических ядер процессора за весь период измерения
5. Пиковая утилизация самого загруженного ядра CPU - Максимальное значение загрузки процессорного ядра ($\geq 80\%$)
6. Максимальная температура CPU за период измерения – Максимальное из замеров температуры датчика CPU за время измерения. ($\leq 60^{\circ}\text{C}$)

7. Температура окружающей среды на входе - Температура воздуха, поступающего в устройство

Задание 4:

В первом задании не использовал ИИ.

ИИ был использован мной в заданиях 2-3, чтобы лучше и быстрее вспомнить или объяснить теоретический материал некоторых курсов, пройденных мной в вузе, который пригодился в этой работе.

Приложение А

Исходный Код

Файл model_no_temp.py (без учета температуры в модели):

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Входные данные из таблицы.
# Формат: [Тактовая частота, Утилизация, Температура]

X_data = np.array([
    [1500, 50, 38],
    [1500, 55, 45],
    [1500, 60, 52],
    [1500, 65, 60],
    [1500, 70, 67],
    [2600, 35, 30],
    [2600, 40, 31],
    [2600, 45, 32],
    [2600, 50, 33],
    [2600, 60, 28]
])

# Целевая переменная: UDP Throughput, Мбит/с.
Y_data = np.array([600, 650, 700, 750, 900, 750, 800, 850, 900, 600])

# Исключаем температуру, так как она является следствием нагрузки.
X_data = X_data[:, :2]

# Обучение модели линейной регрессии.
model = LinearRegression()
model.fit(X_data, Y_data)

# Данные прогноза: [Тактовая частота, Утилизация].
target = np.array([[2000, 40]])
prediction = model.predict(target)

print(f"Прогнозируемый UDP Throughput: {prediction[0]:.2f} Мбит/с")
print(f"Уравнение: Y = {model.coef_[0]:.2f}*Частота + {model.coef_[1]:.2f}*Утилизация + {model.intercept_:.2f}")
```

Файл model_temp.py (с учетом температуры в модели):

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Входные данные из таблицы.
# Формат: [Тактовая частота, Утилизация, Температура]

X_data = np.array([
    [1500, 50, 38],
    [1500, 55, 45],
```

```

        [1500, 60, 52],
        [1500, 65, 60],
        [1500, 70, 67],
        [2600, 35, 30],
        [2600, 40, 31],
        [2600, 45, 32],
        [2600, 50, 33],
        [2600, 60, 28]
    ])

# Целевая переменная: UDP Throughput, Мбит/с.
Y_data = np.array([600, 650, 700, 750, 900, 750, 800, 850, 900, 600])

# Обучение модели линейной регрессии.
model = LinearRegression()
model.fit(X_data, Y_data)

# Данные прогноза: [Тактовая частота, Утилизация, Температура].
target = np.array([[2000, 40, 54]])
prediction = model.predict(target)

print(f"Прогнозируемый UDP Throughput: {prediction[0]:.2f} Мбит/с")
print(f"Уравнение: Y = {model.coef_[0]:.2f}*Частота + {model.coef_[1]:.2f}*Утилизация + {model.coef_[2]:.2f}*Температура + {model.intercept_:.2f}")

```

Файл reverse_regression.py:

```

import numpy as np
from sklearn.linear_model import LinearRegression

# Входные данные из таблицы.
# Формат: [Тактовая частота, Утилизация, Температура]

X_data = np.array([
    [1500, 50, 38],
    [1500, 55, 45],
    [1500, 60, 52],
    [1500, 65, 60],
    [1500, 70, 67],
    [2600, 35, 30],
    [2600, 40, 31],
    [2600, 45, 32],
    [2600, 50, 33],
    [2600, 60, 28]
])

# Целевая переменная: UDP Throughput, Мбит/с.
Y_data = np.array([600, 650, 700, 750, 900, 750, 800, 850, 900, 600])

# Разделяем данные по устройствам
X_var1 = X_data[:5, :]
X_var2 = X_data[5:, :]
Y_var1 = Y_data[:5]

```

```

Y_var2 = Y_data[5:]

# Модели для утилизации устройства 1 и 2
model_cpu1 = LinearRegression()
model_cpu1.fit(Y_var1.reshape(-1, 1), X_var1[:, 1])

model_cpu2 = LinearRegression()
model_cpu2.fit(Y_var2.reshape(-1, 1), X_var2[:, 1])

# Модели для температуры устройства 1 и 2
model_temp1 = LinearRegression()
model_temp1.fit(Y_var1.reshape(-1, 1), X_var1[:, 2])

model_temp2 = LinearRegression()
model_temp2.fit(Y_var2.reshape(-1, 1), X_var2[:, 2])

# Функция интерполяции для частоты 1700 ГГц
def interpolate_for_freq(throughput):
    # Данные частоты
    freq4 = 1700
    freq1, freq2 = 1500, 2600
    # Получение предсказываемых значений для введенной частоты для устройств 1 и 2
    cpu1 = model_cpu1.predict([[throughput]])[0]
    temp1 = model_temp1.predict([[throughput]])[0]
    cpu2 = model_cpu2.predict([[throughput]])[0]
    temp2 = model_temp2.predict([[throughput]])[0]
    # Коэффициент интерполяции
    k = (freq4 - freq1) / (freq2 - freq1)
    # Утилизация и температура для новой введенной частоты
    # Рассчитывается на основе данных устройств 1 и 2, а также их схожести с четвертым
    new_cpu = cpu1 + k * (cpu2 - cpu1)
    new_temp = temp1 + k * (temp2 - temp1)

    return new_cpu, new_temp

# Коррекция для малых значений Throughput
def correct_predictions(throughput):
    cpu_600, temp_600 = interpolate_for_freq(600)

    if throughput <= 600:
        cpu_corr = (throughput / 600) * cpu_600
        temp_corr = 25 + (throughput / 600) * (temp_600 - 25)
    else:
        cpu_corr, temp_corr = interpolate_for_freq(throughput)

    cpu_corr = max(0, min(100, cpu_corr))
    temp_corr = max(20, min(100, temp_corr))

    return cpu_corr, temp_corr

# Заданные значения UDP Throughput из таблицы 2
throughputs = [50, 100, 150, 200, 250, 300, 350, 400, 450, 500]

# Формирование таблицы результатов

```

```
results = []
for tput in throughputs:
    new_fin_cpu, new_fin_temp = correct_predictions(tput)
    results.append((tput, new_fin_cpu, new_fin_temp))

print(f'UDP Throughput:<15} {'Утилизация ЦП':<15} {'Температура ЦП':<15}")
for tput, cpu, temp in results:
    print(f"{tput:<15} {cpu:<15.1f} {temp:<15.1f}")
```